

Burjex Prime Backend - Audit & Verification Checklist ANSWERS

Read-only audit (no code changes). Date: 2026-09-10

Repo: Burjex-complete-project | Live target: burjexprime.net (burjex_net)

KEY FINDING: No Rust backend. Live trading = Node.js/TypeScript btrader + Django CRM.

Checklist answered vs ACTUAL stack; Rust items marked NO with ALTERNATES where present.

IMPORTANT PREFACE

ITEM: Stack reality

STATUS: ABSENT (as Rust)

ANSWER: NO Rust backend exists in this repo (0 Cargo.toml / 0 .rs files).

ALTERNATE / EVIDENCE: Live trading stack is Node.js 20+ TypeScript (btrader/) + Django CRM.

Checklist items answered against ACTUAL live stack, with Rust expectation marked and alternates noted.

1. Concurrency & Thread Safety (Tokio / Rust Specific)

ITEM: Async Runtime (Tokio)

STATUS: NO (Tokio)

ANSWER: Backend does NOT run on Tokio.

ALTERNATE / EVIDENCE: ALTERNATE: Node.js V8 event loop + async/await (trading-engine main.ts, NestJS gateway). I/O is non-blocking via Prisma/ioredis Promises.

ITEM: Thread Safety & Shared State (Arc/Mutex/DashMap)

STATUS: NO (Rust primitives)

ANSWER: No Arc<Mutex>, RwLock, or DashMap.

ALTERNATE / EVIDENCE: ALTERNATE / PARTIAL: In-process JS Map books (position-book.ts, pending-book.ts, price-source.ts) + Redis SET NX instance lock (instance-lock.ts) so only one engine process owns ticks. Single-threaded JS avoids classic data races; multi-gateway in-process books can drift.

ITEM: Actor Model / Message Passing (Actix / mpsc)

STATUS: NO (Actix/mpsc)

ANSWER: No Actix actors or Tokio mpsc queues.

ALTERNATE / EVIDENCE: ALTERNATE / PARTIAL: Redis Streams (bt:engine:cmd XREADGROUP) + Redis pub/sub for ticks. REST opens go in-process via gateway TradingEngine.placeOrder() - not a single global actor mailbox for all fills.

2. Order Matching Engine & Performance

ITEM: In-Memory Matching

STATUS: PARTIAL (dealer model)

ANSWER: Not a central limit order book (LOB) matcher.

ALTERNATE / EVIDENCE: ALTERNATE PRESENT: In-RAM position book, pending book, L1 bid/ask cache. Fills are MT5-like dealer execution vs LP/broker quote (A/B-book routing), not peer matching.

ITEM: Sub-Millisecond Processing

STATUS: NO

ANSWER: No sub-ms matching architecture.

ALTERNATE / EVIDENCE: ALTERNATE / DESIGN TARGET: Docs target low tens of ms p99 intra-region. Tick drains default 50ms/250ms; MARKET mode may add executionDelayMs. Hot path is sync REST + Prisma transaction.

ITEM: FIFO Queue Implementation

STATUS: PARTIAL

ANSWER: No strict LOB price-time FIFO match queue.

ALTERNATE / EVIDENCE: ALTERNATE: Redis Stream commands are FIFO; CRM outbox ordered by createdAt; stop-out tie-break uses openedAt. Pending trigger walks Map (not a dedicated FIFO fill queue).

3. Database & Data Persistence

ITEM: Zero Blocking DB Operations

STATUS: PARTIAL / NO on money path

ANSWER: Order open/close use synchronous Prisma DB transactions on the critical path.

ALTERNATE / EVIDENCE: ALTERNATE PRESENT for ancillary work: CRM outbox worker (~2s), floating P/L flush (PROFIT_PERSIST_MS), candle history queue, swap accrual; Django Celery for CRM only (not order matching).

ITEM: Database Sharding / Partitioning

STATUS: NO (physical)

ANSWER: No PostgreSQL PARTITION BY / trade table sharding in live schema.

ALTERNATE / EVIDENCE: ALTERNATE / PARTIAL: Logical tenantId isolation; optional ENGINE_SHARD hash in code; live burjexprime compose runs single bxnet_bt_postgres + 1 engine replica. Docs mention future dedicated DB / K8s sharding.

ITEM: In-Memory Caching (Redis)

STATUS: YES (ticks) / PARTIAL (sessions)

ANSWER: Redis is active for realtime trading data.

ALTERNATE / EVIDENCE: PRESENT: lastticks hash, tick pub/sub, engine command streams, L1 price Map. CRM uses Redis-backed cache/sessions in prod. Trading API auth is JWT (stateless), not Redis sessions.

4. Network & Latency Management

ITEM: Low-Latency Networking (serde / tungstenite)

STATUS: NO (Rust net stack)

ANSWER: No serde zero-copy or tokio-tungstenite.

ALTERNATE / EVIDENCE: ALTERNATE PRESENT: ws package WebSocket gateway + NestJS REST + JSON. Tick batching (WS_TICK_BATCH_MS). Latency helpers in packages/shared. Not MessagePack/Protobuf.

ITEM: Co-location / Infrastructure Ready

STATUS: PARTIAL

ANSWER: Clean Docker Compose deploy exists; not colo-tuned HFT infra.

ALTERNATE / EVIDENCE: PRESENT: docker-compose.github.yml, Caddy edge, multi-stage Docker builds. ABSENT live: Equinix-style colo, CPU pinning, multi-AZ HA. K8s manifests exist as aspirational, not live burjexprime topology.

5. Risk Control & Admin Group Settings

ITEM: Strict Group Rules Enforcement

STATUS: YES

ANSWER: Admin group/symbol rules enforced in engine-core.

ALTERNATE / EVIDENCE: PRESENT: spreads/markup, leverage/margin, symbol sessions, max lot/positions, stop-out/margin call, Instant/Market executionApplyTo + delay. Admin Flutter groups UI.

ITEM: Slippage & Requote Logic

STATUS: YES (slippage/Instant-Market) / PARTIAL (requote)

ANSWER: Automated slippage and Instant/Market modes are active.

ALTERNATE / EVIDENCE: PRESENT: slippagePoints, MARKET delay, INSTANT honour. REQUOTE error

code exists but engine does not currently throw requote on deviation (legacy/unused path).

6. Stress, Load & Chaos Testing

=====

ITEM: Load Testing Script (K6 / Locust)

STATUS: NO

ANSWER: No k6/Locust/Artillery load suite found for trading.

ALTERNATE / EVIDENCE: ALTERNATE: Jest unit tests only (calc/trigger/stale). Roadmap docs list load tests as future work.

ITEM: Memory Leak Check (Valgrind / profiling)

STATUS: NO (Valgrind) / PARTIAL (metrics)

ANSWER: No Valgrind or continuous heap-leak suite.

ALTERNATE / EVIDENCE: ALTERNATE PARTIAL: in-process latency stats, Prometheus /v1/metrics text, optional Sentry env hook documented - not a full memory-leak verification program.

7. Security, Error Handling & High Availability

=====

ITEM: Panic-Free Code (unwrap/expect / Result)

STATUS: N/A (not Rust) - ALTERNATE YES pattern

ANSWER: No Rust unwrap/expect/panic model.

ALTERNATE / EVIDENCE: ALTERNATE: TypeScript BtError + BtErrorCode; Nest filter maps to HTTP. Tick loops catch/log. Some fire-and-forget .catch on non-critical Redis publishes.

ITEM: Redundancy & Failover

STATUS: PARTIAL / mostly NO on live

ANSWER: Live burjexprime stack is effectively single-instance per service.

ALTERNATE / EVIDENCE: PARTIAL PRESENT: Redis engine instance lock (loser exits). Docs/K8s describe replicas, Patroni, Redis Sentinel - not deployed on current compose. No instant secondary failover proven live.

ITEM: Bank-Level Encryption (TLS + JWT/OAuth)

STATUS: YES (TLS+JWT) / NO (OAuth on trading)

ANSWER: Edge TLS + trading JWT are in place.

ALTERNATE / EVIDENCE: PRESENT: Caddy/nginx TLS termination; access/refresh JWT; WS ?token=; CRM webhook HMAC. ABSENT on live trading path: OAuth. Social-login CRM code is local leftover and excluded from live deploy by policy.

=====

SUMMARY SCORECARD

- Rust/Tokio/Actix/DashMap/tungstenite/Valgrind LOB matching: NOT PRESENT
- Present equivalents: Node async, Redis streams/pubsub, in-memory dealer books, group risk rules, Instant/Market+slippage, Redis ticks, JWT+TLS, Docker deploy
- Gaps vs checklist ideal: sub-ms matching, physical DB shards, load tests, live HA failover, active requote throws, OAuth on trading API

Evidence roots: btrader/apps/trading-engine, btrader/packages/engine-core, btrader/apps/gateway-api, btrader/apps/ws-gateway, btrader/apps/market-data, deploy/docker-compose.github.yml, btrader/docs/01-system-architecture.md

END OF AUDIT - read-only; no repository code was modified for this document.